

TrustOps AI Platform

Projeto Executivo de Implementação◆◆◆◆o

Projeto Executivo de Implementação — TrustOps AI Platform

1. Visão Executiva
 2. Problema que o produto resolve
 3. Objetivos do projeto
 4. Arquitetura técnica completa
 5. Arquitetura de componentes
 6. Trust Engine — o núcleo do produto
 7. Stack tecnológica
 8. Estrutura do repositório GitHub
 9. Backlog executivo de sprints
 10. Primeiro MVP funcional
 11. Fluxo de uso do MVP
 12. Modelo inicial de dados
 13. Critérios de sucesso do MVP
 14. Roadmap de evolução
- Recomendação de execução

Projeto Executivo de Implementação — TrustOps AI Platform

1. Visão Executiva

Nome do projeto

TrustOps AI Platform

Proposta

Uma plataforma de **DevSecOps, Software Quality e AI Trust** capaz de avaliar automaticamente aplicações e sistemas de IA, consolidando evidências técnicas e produzindo um **Trust Score** e uma **certificação interna de confiança** por versão do software.

O produto transforma este processo:

"Eu acredito que meu software é seguro."

neste:

"Esta versão foi avaliada por controles automatizados, possui evidências rastreáveis, riscos identificados e atingiu os critérios definidos para o nível de certificação."

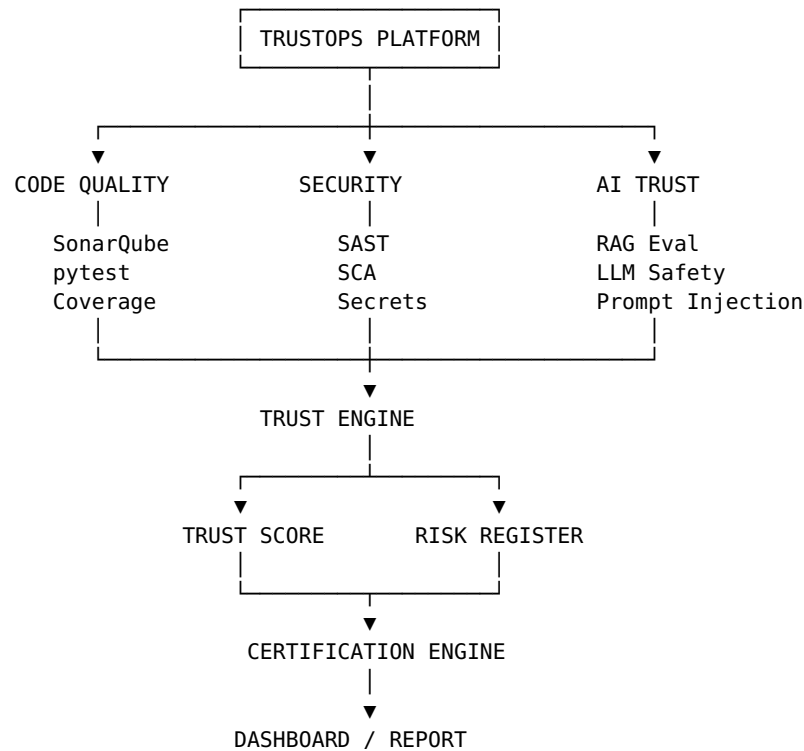
2. Problema que o produto resolve

Em muitos projetos, qualidade, segurança e confiabilidade são avaliadas por ferramentas isoladas:

SonarQube → Qualidade
 Trivy → Containers
 Bandit → Python Security
 pytest → Testes
 OWASP → Segurança
 RAGAS → Qualidade RAG

O problema é que o desenvolvedor precisa interpretar diversos relatórios.

A **TrustOps AI Platform** centraliza essas informações:



3. Objetivos do projeto

Objetivo principal

Construir uma plataforma capaz de avaliar automaticamente um projeto de software e responder:

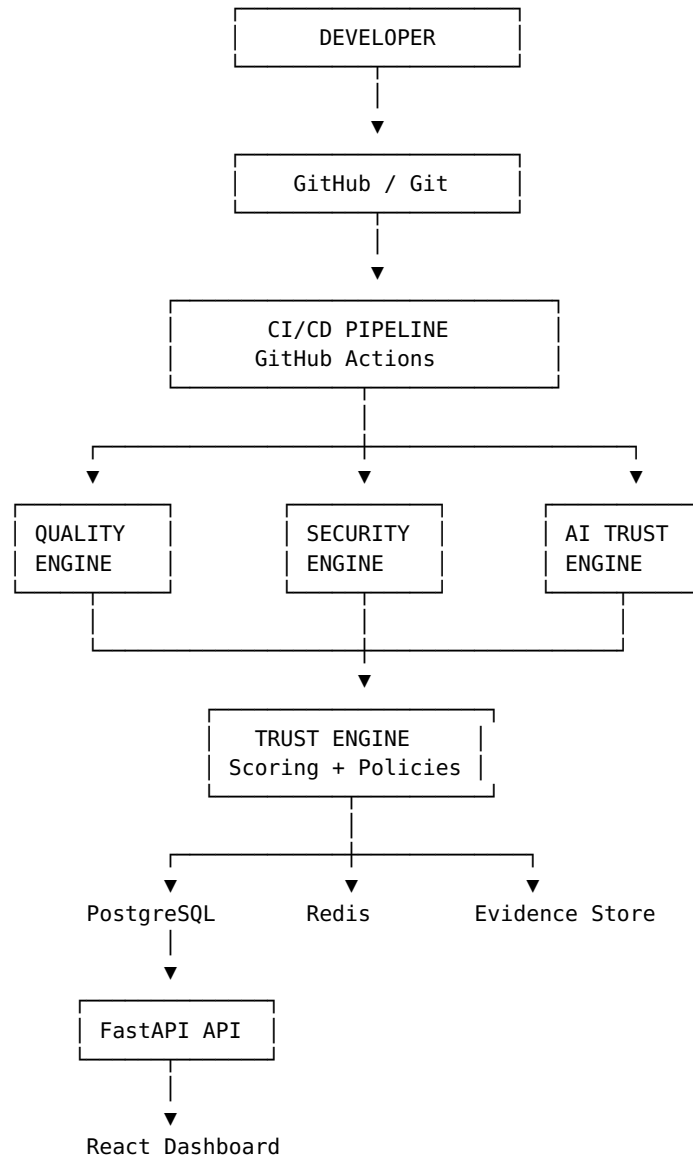
- Qual é o nível de segurança?
- Quais vulnerabilidades existem?
- Qual é a qualidade do código?
- O software possui testes suficientes?
- Quais riscos impedem a publicação?
- Qual é o nível de confiança da versão?
- O software pode receber uma certificação interna?

Objetivos técnicos

- Automatizar coleta de evidências.
- Criar um motor de cálculo do Trust Score.
- Implementar políticas de **Quality Gates**.
- Manter histórico de avaliações.
- Criar dashboard de riscos.
- Gerar relatórios de certificação.
- Preparar a arquitetura para avaliação de IA.

4. Arquitetura técnica completa

Arquitetura de alto nível



5. Arquitetura de componentes

5.1 Backend — TrustOps API

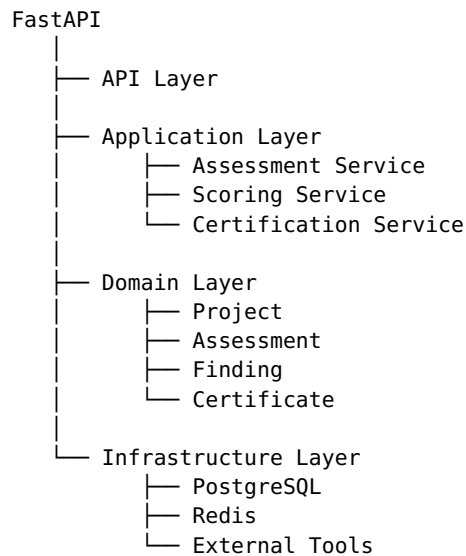
Responsabilidades:

- autenticação;
- cadastro de projetos;
- execução de avaliações;
- consulta de resultados;
- gestão de riscos;
- cálculo do Trust Score;
- geração de certificados.

Tecnologia:

Python + FastAPI

Estrutura interna:



A arquitetura recomendada é **Clean Architecture + Domain-Driven Design simplificado**.

6. Trust Engine — o núcleo do produto

O Trust Engine consolida os resultados de todas as ferramentas.

Entrada

```
{
  "security": {
    "critical": 0,
    "high": 2,
    "medium": 5
  },
  "quality": {
    "coverage": 82,
    "bugs": 3
  },
  "reliability": {
    "availability": 99.7,
    "error_rate": 0.3
  }
}
```

Processamento

O motor executa:

1. normalização das métricas;
2. aplicação dos pesos;
3. aplicação de penalidades;
4. verificação de bloqueadores;
5. cálculo do score;
6. definição do nível de certificação.

Pseudológica

```
IF vulnerabilities.critical > 0:
```

```
Certification = BLOCKED

ELSE:
    Score = CalculateWeightedScore()

    IF Score >= 95:
        Level = ENTERPRISE_TRUST

    ELIF Score >= 85:
        Level = HIGH_TRUST

    ELIF Score >= 75:
        Level = TRUSTED

    ELSE:
        Level = FOUNDATION
```

7. Stack tecnológica

Backend

Tecnologia	Função
Python 3.12+	Linguagem principal
FastAPI	API REST
Pydantic	Validação
SQLAlchemy	ORM
Alembic	Migrations
Celery	Processamento assíncrono
Redis	Broker/cache
PostgreSQL	Banco de dados

Qualidade

Tecnologia	Função
pytest	Testes
Ruff	Linting
mypy	Type checking
coverage.py	Cobertura
SonarQube	Qualidade e análise

Segurança

Tecnologia	Função
Bandit	Segurança Python
Semgrep	SAST
Trivy	Container e dependências
Gitleaks	Secrets
OWASP ZAP	DAST

IA

Tecnologia	Função
RAGAS	Avaliação de RAG

DeepEval	Avaliação de LLM
LangChain/LangGraph	Orquestração futura
MLflow	Rastreamento de avaliações
PostgreSQL + pgvector	Dados vetoriais iniciais

Infraestrutura

Tecnologia	Função
Docker	Containers
Docker Compose	Ambiente local
GitHub Actions	CI/CD
Terraform	Infraestrutura como código
Kubernetes	Evolução futura
Prometheus	Métricas
Grafana	Observabilidade

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- Recharts

8. Estrutura do repositório GitHub

Recomendo um **monorepo**, especialmente para o MVP.

```
trustops-ai-platform/
├── README.md
├── LICENSE
├── CONTRIBUTING.md
├── SECURITY.md
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── api/
│   └── trust-framework/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── core/
│   │   ├── domain/
│   │   ├── application/
│   │   ├── infrastructure/
│   │   └── main.py
│   ├── tests/
│   └── requirements/
├── frontend/
│   ├── src/
│   └── public/
├── trust-engine/
│   └── scoring/
```

```
| policies/
| certification/
| tests/
|
| integrations/
|   semgrep/
|   trivy/
|   gitleaks/
|   pytest/
|
| infrastructure/
|   docker/
|   terraform/
|   kubernetes/
|
| evidence/
|   README.md
|
| .github/
|   workflows/
|     ci.yml
|     security.yml
|     release.yml
|
| docker-compose.yml
```

9. Backlog executivo de sprints

📅 SPRINT 0 — Foundation & Architecture

Objetivo: estabelecer a fundação.

Entregas

- ☐ Criar repositório
- ☐ Definir arquitetura
- ☐ Criar ADRs
- ☐ Configurar Docker
- ☐ Configurar PostgreSQL
- ☐ Configurar CI inicial
- ☐ Criar documentação do STSA

Resultado: ambiente de desenvolvimento funcionando.

🚀 SPRINT 1 — Core Platform

Objetivo: construir o núcleo.

Entregas

- ☐ FastAPI
- ☐ Health check
- ☐ Modelo de Project
- ☐ Modelo de Assessment
- ☐ PostgreSQL
- ☐ Alembic migrations
- ☐ CRUD de projetos

Resultado: API funcional.

🛡️ SPRINT 2 — Quality Assessment

Entregas

- ☐ Integração pytest
- ☐ Coleta de cobertura
- ☐ Integração Ruff
- ☐ Modelo de Quality Findings
- ☐ Quality Score

Resultado: avaliação automatizada de qualidade.

🛡️ SPRINT 3 — Security Assessment

Entregas

- ☐ Semgrep
- ☐ Bandit
- ☐ Gitleaks
- ☐ Trivy
- ☐ Normalizador de findings
- ☐ Security Score

Resultado: primeira camada DevSecOps.

🧠 SPRINT 4 — Trust Engine

Entregas

- ☐ Sistema de pesos
- ☐ Sistema de penalidades
- ☐ Quality Gates
- ☐ Trust Score
- ☐ Risk Classification

Resultado: decisão automatizada de aprovação.

📊 SPRINT 5 — Dashboard MVP

Entregas

- ☐ Dashboard
- ☐ Lista de projetos
- ☐ Trust Score
- ☐ Histórico
- ☐ Riscos críticos

Resultado: visualização executiva.

🔐 SPRINT 6 — Certification Engine

Entregas

- ☐ Níveis de certificação
- ☐ Certificado por versão
- ☐ Relatório de avaliação
- ☐ Evidências
- ☐ Risk Register

Resultado: certificação interna.

🏠 SPRINT 7 — AI Trust Assessment

Entregas

- ☐ Dataset de avaliação
- ☐ Avaliação de RAG
- ☐ Hallucination testing
- ☐ Prompt Injection tests
- ☐ AI Trust Score

Resultado: diferenciação do projeto.

🏠 SPRINT 8 — Production & Observability

Entregas

- ☐ Prometheus
- ☐ Grafana
- ☐ Logs estruturados
- ☐ Alertas
- ☐ Terraform
- ☐ Deploy cloud

Resultado: plataforma production-ready.

10. Primeiro MVP funcional

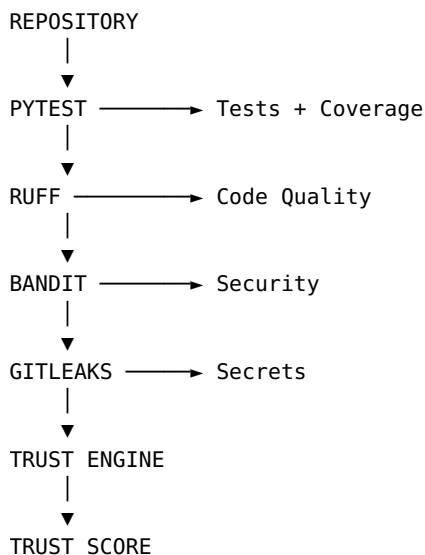
Para evitar construir uma plataforma excessivamente grande no início, o MVP deve ser objetivo.

🏠 MVP v0.1

Entrada

Um repositório Python.

Avaliações



↓
CERTIFICATION

Saída

```
{  
  "project": "example-api",  
  "version": "0.1.0",  
  "trust_score": 87,  
  "certification": "HIGH_TRUST",  
  "status": "APPROVED",  
  "findings": {  
    "critical": 0,  
    "high": 1,  
    "medium": 3  
  }  
}
```

11. Fluxo de uso do MVP

1. Usuário cadastra projeto
 2. TrustOps recebe URL do repositório
 3. Assessment Job é criado
 4. Ferramentas executam scans
 5. Resultados são normalizados
 6. Trust Engine calcula score
 7. Quality Gate decide:
 PASS / FAIL
 8. Certificado é emitido
-

12. Modelo inicial de dados

PROJECT

```
|  
├─ id  
├─ name  
├─ repository_url  
└─ created_at
```

ASSESSMENT

```
|  
├─ id  
├─ project_id  
├─ version  
└─ status
```

└─ trust_score

FINDING

├─ id
├─ assessment_id
├─ tool
├─ severity
├─ category
└─ description

CERTIFICATE

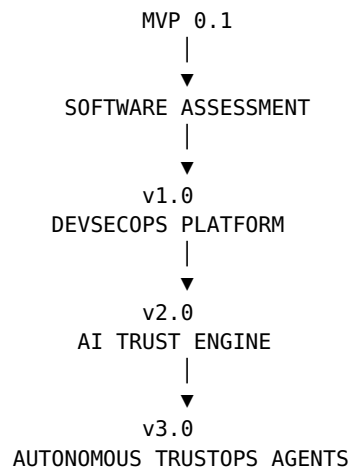
├─ id
├─ assessment_id
├─ certification_level
├─ status
└─ issued_at

13. Critérios de sucesso do MVP

O MVP será considerado concluído quando conseguir:

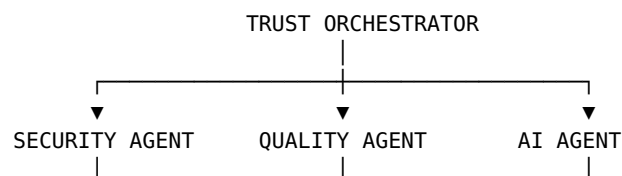
- ✓ Avaliar um repositório real.
 - ✓ Executar pelo menos quatro ferramentas automatizadas.
 - ✓ Consolidar resultados diferentes.
 - ✓ Calcular o Trust Score.
 - ✓ Aplicar um Quality Gate.
 - ✓ Persistir resultados.
 - ✓ Exibir o resultado via API.
 - ✓ Emitir uma certificação interna.
-

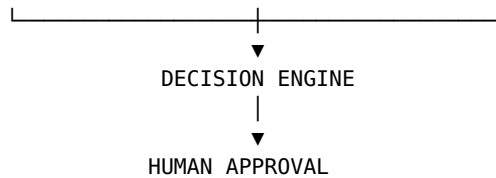
14. Roadmap de evolução



Fase futura: Agentes autônomos

Essa evolução conversa diretamente com sistemas de agentes:





O **Human-in-the-Loop** permanece para decisões de alto impacto, como aprovar exceções de risco ou liberar software crítico.

Recomendação de execução

Começar imediatamente pelo **MVP v0.1**, com foco em um repositório Python e quatro pilares concretos: **testes, qualidade, segurança e Trust Score**. A maior inovação do projeto não é simplesmente integrar ferramentas: é construir um **modelo consistente de normalização, decisão, evidências e certificação**.

Este projeto tem potencial de se tornar um dos projetos centrais de portfólio por demonstrar, de forma integrada, competências de **Arquitetura de Software, Engenharia de Dados, DevSecOps, Python, APIs, Docker, CI/CD e Engenharia de IA**.